$SO(3)$ Gauge Theory in $\mathcal{UD}$

Brian Beckman

Mar 2026

Abstract

In this notebook, we re-analyze the inverted spherical pendulum in new terms: as a gauge theory in $SO(3)$, the Lie group of rotations in 3 dimensions. We first reproduce the equations of motion in pitch/-cone-roll coordinates, known from earlier notebooks, from connections in the covariant derivative, computed automatically from a kinetic-energy metric. This metric measures distances between configuration velocities in the 2D quotient group $SO(3)/SO(2)$. Adding a third angle ψ, spinning the pendulum on its long axis, extends the group via *fibration* to $SO(3)$. Physically, it adds gyroscopic forces to the pendulum, making it easier to stabilize than by, say, control forces applied at the base. We offer an interactive visualization that lets a user smoothly vary the system from a free pendulum that falls, to a gyroscope that precesses and nutates. The visualization also reveals coordinate singularities, which we eliminate in an upcoming notebook that addresses the problem in another gauge, $SU(2)$, the unitary group of spinors or quaternions, the exact same group used in quantum theory of the Standard Model.

Plan for this Notebook

- Recap the established Lagrangian formulation and visualization of the inverted spherical pendulum (ISP) in pitch/cone-roll coordinates.
- Reformulate in geometric terms, with kinetic energy as the metric tensor, and potential energy as a scalar gradient field.
  - Do this twice to make a software-engineering point: once in the toolkit's newer "indexer sector," and again in the older "$\mathcal{UD}$ sector" via an existing gradient operator.
- Show that the equations of motion are identical however derived.
- Exhibit breakdown of the numerical solution at a singular point of the coordinates, setting the stage for upcoming resolution in the $SU(2)$ gauge.
- Add the third angle—twist around the baton axis—adding gyroscopic effects to the physics. Visualize the results as before.

Initialize the Relativity Toolkit

Optional: Quit for a Fresh Kernel

- *Note: if you call* **Quit** *in a notebook, evaluation may hang. If you want to "Evaluate Notebook," mark the following two cells "Evaluatable" in the Cell→Property menu, call* **Quit** *and* **FrontEndTokenExecute** *once each, then comment out the cells, or mark them unevaluatable again. Alternatively, you can quit the kernel via the Evaluation menu (last item). At that point, you can "Evaluate Notebook" in a clean environment, or evaluate cells individually.*

```
Quit[];
```

In[•]:=
```
FrontEndTokenExecute["DeleteGeneratedCells"]
```

Download the Code

In[1]:=
```
getToolkit[branch_] :=
  With[
   {base = "https://raw.githubusercontent.com/rebcabin/RelativityToolkit/" <>
      branch <> "/",
    bust = "?t=" <> ToString[Round[AbsoluteTime[]]]},
   {rules = base <> "RelativityToolkit.wl" <> bust},
   Get[rules]];
getToolkit["v1.13.0"];
```

» Relativity Toolkit version : 1.13.0

» Relativity Connection Symbol : Γ

Original Problem, Visualization, and Solution

This section is a recap of the standard Euler-Lagrange process, which was fully developed in *Perspectives on energy conservation: discrete variational spherical pendulum*, April 2025. There are three subsections here, copy-pasted and lightly edited from that notebook:

- Save and print equations of motion for later comparison with the new geometric process.
- Visualize the 2D, $\{\delta, \eta\}$, $SO(3)/SO(2)$ coordinate system, common to both the old Euler-Lagrange process and to the new geometric derivation.
- Visualize solutions of the equations, pertinent to both derivation processes.

Equations of Motion via Euler-Lagrange

Here are the old Euler-Lagrange definitions without the explanations that appear in the old notebook. All we really need from this is the **Echo** of the Euler-Lagrange equations at the end and the setups for the visualizations to follow.

In[3]:=
```
ClearAll[e1, e2, e3, o, g, plotRg];
e1 = {1, 0, 0}; e2 = {0, 1, 0}; e3 = {0, 0, 1};
```

```
o = {0, 0, 0}; g = 9.81; plotRg = {-1.25, 1.25};

ClearAll[cm];
(cm[{δ_, η_}] :=
   RotationMatrix[η, e1] . RotationMatrix[δ, e2] . {0, 0, Subscript[c, z]});

ClearAll[zh];
zh[{δ_, η_}] := cm[{δ, η}][[3]];

ClearAll[numRules];
numRules = {δ[t] → δ, η[t] → η, δ'[t] → Dδ, η'[t] → Dη, m → 1, Subscript[c, z] → 1.2};

ClearAll[Vnum, V];
V[{δ_, η_}] := m g zh[{δ, η}];
Vnum[{δ_, η_}] = ((V[{δ, η}]) /. numRules);

ClearAll[v];
(v[{δ_, η_}] := D[cm[{δ, η}], t]);

ClearAll[Tnum, T];
(T[{δ_, η_}] := 1 / 2 m v[{δ, η}] . v[{δ, η}]);
Tnum[{δ_, η_}, {Dδ_, Dη_}] := 1 / 2 (1.2) ^ 2 (Dδ ^ 2 + (1 / 2 (1 + Cos[2 δ])) Dη ^ 2);
Tnum[{δ_, η_}, {Dδ_, Dη_}] = (T[{δ[t], η[t]}] /. numRules // FullSimplify);

ClearAll[Lnum, L];
(L[{δ_, η_}] := T[{δ, η}] - V[{δ, η}]);

ClearAll[Leqns];
With[{Lcrds = {δ[t], η[t]}},
  With[{Lvels = D[Lcrds, t]},
   With[{Lncnf = {0, 0}}, (* non-conservative forces *)
    Leqns = Flatten@MapThread[
        {q, v, f} ↦ D[D[L[Lcrds], v], t] - D[L[Lcrds], q] == f,
        {Lcrds, Lvels, Lncnf}]]]];

ClearAll[stEqns];
(stEqns =
   Equal @@@ (Solve[Leqns, {δ''[t], η''[t]}] // FullSimplify // Chop // Part[#, 1] &));
Echo[stEqns // TableForm, "State-space equations (stEqns) :"];

ClearAll[tLim, initialConditions, stSolns];
tLim = 200;
```

» State–space equations (stEqns) :

$$\delta''[t] == \text{Sin}[\delta[t]]\left(\frac{9.81\,\text{Cos}[\eta[t]]}{c_z} - 1.\,\text{Cos}[\delta[t]]\,\eta'[t]^2\right)$$
$$\eta''[t] == \frac{9.81\,\text{Sec}[\delta[t]]\,\text{Sin}[\eta[t]]}{c_z} + 2.\,\text{Tan}[\delta[t]]\,\delta'[t]\,\eta'[t]$$

- *The second, η'', equation has* `Sec[δ[t]]`*, which equals* `ComplexInfinity` *when* $\delta = (\pi/2) = 90°$. *That's the source of divergence we'll see below in the solutions.*

Visualization 1: The Coordinate System

The standard spherical coordinates are singular at the North and South poles, at the two equilibrium positions of the ISP—hanging straight down or balanced straight up. In neither of these positions is azimuth mathematically defined. These coordinates are problematic for computation.

- *I witnessed an accident wherein the control software for a radio telescope failed to account for the singularity, wrapped cables around the azimuthal axis mount, and stretched them to breaking. No one was hurt.*

Turn the spherical coordinate system 90° from vertical about the Y axis so that the poles are on the geophysical Equator. By so moving the poles, the equilibrium positions are no longer singular. We haven't removed the singularity, merely moved it from the equilibrium positions, hiding it for the common equilibrium cases (specifically for our earlier control applications).

Moving the singularity does not remove it, however. Topologically, no smooth, 2D coordinate grid can cover a sphere without a singularity. When the baton falls across the equator at $\delta = (\pi/2) = 90°$, the η degree of freedom becomes undefined and the math breaks as `Sec[δ[t]]`, in the η'' equation, goes to `ComplexInfinity`. This topological singularity is inevitable in any global coordinate system on $SO(3)$ (See references to Do Carmo, Marsden et al., Baez and Muniain, and Burke).

Measure the angular position of the baton with two angles:

- δ, measuring co-latitude from North. Call it ***pitch***.
- η, measuring angle counterclockwise around the original X axis. Call it ***cone-roll***.

Both angles can endure adding or subtracting any multiple of 360°.

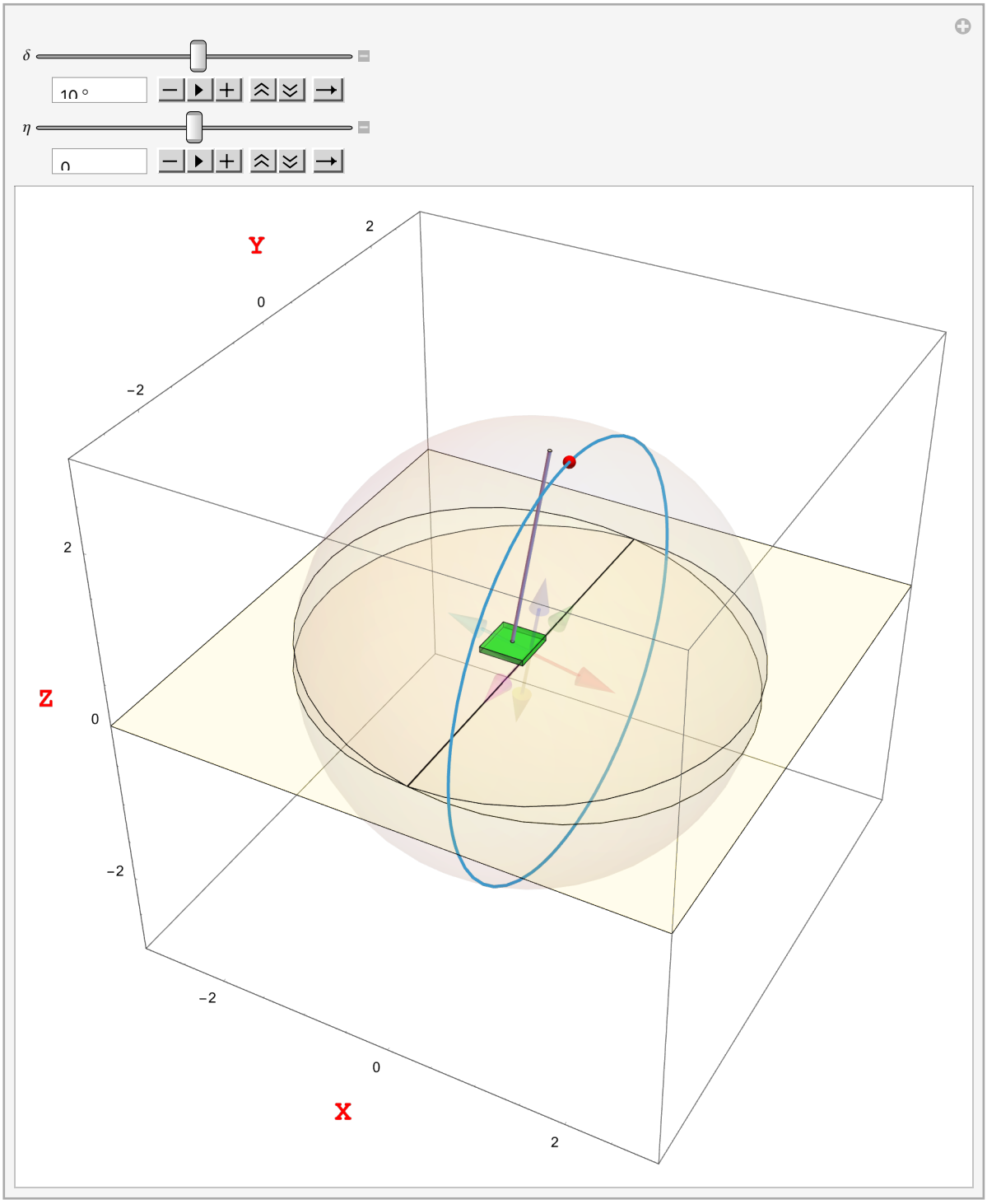
This global coordinate system will be as good as any in $SO(3)$ or $SO(3)/SO(2)$ for numerical integration.

- *One way out of the singularity trap is multiple coordinate systems with transition functions in overlap areas. That's a good solution, but much more complicated than going to* $SU(2)$*, which double-covers the sphere with no singularity. When we go to* $SU(2)$ *in an upcoming notebook, there will be no poles to worry about.*

In[30]:=
```
ClearAll[rig];
With[{l = 1.2, mass = 1.0, d = 2.4, r = 0.02},
  {baton = Cylinder[{{0, 0, -l}, {0, 0, +l}}, r],
   batonCenterBodyFrame = {0, 0, l}},
  rig["graphics primitives"] = baton;
  rig["moment of inertia"] =
   With[{Ib = mass * (MomentOfInertia[baton] / Volume[baton])},
    {Ibi = Inverse@Ib},
```

```
    {Ib, Ibi}];
  rig["cb"] := batonCenterBodyFrame;
  rig["half length"] := l;
  rig["mass"] := mass;];
ClearAll[jack];
jack[0, opacity_ : 0.1, diameter_ : 0.01] := {
   Opacity[opacity],
   {Red, Arrow[Tube[{o, e1}], diameter]},
   {Darker[Green], Arrow[Tube[{o, e2}], diameter]},
   {Lighter[Blue], Arrow[Tube[{o, e3}], diameter]},
   {Lighter[Cyan], Arrow[Tube[{o, -e1}], diameter]},
   {Magenta, Arrow[Tube[{o, -e2}], diameter]},
   {Yellow, Arrow[Tube[{o, -e3}], diameter]}};
With[{epsilon = 0.0001, h = 1 / 15., w = 1 / 4., vu = 3., o = {0, 0, 0},
  thin = {0, 0, .0001},
  cb = rig["cb"]},
 {kart = Cuboid[{-w, -w, epsilon}, {w, w, epsilon + h}], s = 2,
  floor = Polygon[{{-vu, -vu, 0}, {vu, -vu, 0}, {vu, vu, 0}, {-vu, vu, 0}}]},
 {c3 = Cylinder[{o, thin}, s cb[[3]]],
  b3 = Sphere[o, s cb[[3]]], t3x = Tube[{-s cb[[3]] e2, s cb[[3]] e2}, .01],
  axisLabelStyle = text ↦ Style[text, Red, Bold, 18, FontFamily → "Courier New"]},
 Manipulate[
  DynamicModule[
   {renderPos = o, Στs = o, cs, rotfn},
   cs = RotationMatrix[η, e1].RotationMatrix[δ, e2].cb;
   renderPos = -cs[[1]] e1 - cs[[2]] e2;
   rotfn = RotationTransform[η, e1] @* RotationTransform[δ, e2];
   Show[{Graphics3D[{GeometricTransformation[jack[0], rotfn],
       {Black, t3x},
       {White, Opacity[.7 / 8], c3, b3,
        GeometricTransformation[c3, RotationTransform[δ, e2]]},
       {Red, Sphere[s cs, 1 / 16.]},
       {Yellow, Opacity[.3 / 4], floor},
       {Green, Opacity[.6], Translate[kart, renderPos]}, {White, Opacity[.75],
        Translate[
         Translate[
          GeometricTransformation[rig["graphics primitives"], rotfn], cs],
         renderPos + (epsilon + h) e3]}},
      ImageSize → Large, Axes → True, AxesLabel → axisLabelStyle /@ {"X", "Y", "Z"},
      PlotRange → {{-vu, vu}, {-vu, vu}, {-vu, vu}}],
     ParametricPlot3D[
      s RotationMatrix[ha, e1].RotationMatrix[δ, e2].cb, {ha, 0, 2 π}]}]],
  {{δ, 10 °}, -360 °, 360 °, 1 °, Appearance → {"Open"}},
  {{η, 0 °}, -360 °, 360 °, 1 °, Appearance → {"Open"}}]]
```

Out[34]=



Visualization 2: Solving the Equations of Motion

The next **Manipulate** lets you drop the baton from any initial conditions for δ and η. Gravity takes over, swinging the pendulum down and back up on the other side, then down again and back up on the starting side, etc. The heads-up display (HUD) shows the conserved values of energy $E = T + V$, and the Z

component of angular momentum p_z (formula derived below). This Z component of angular momentum is zero for all settings of the initial conditions because gravity points straight down Z.

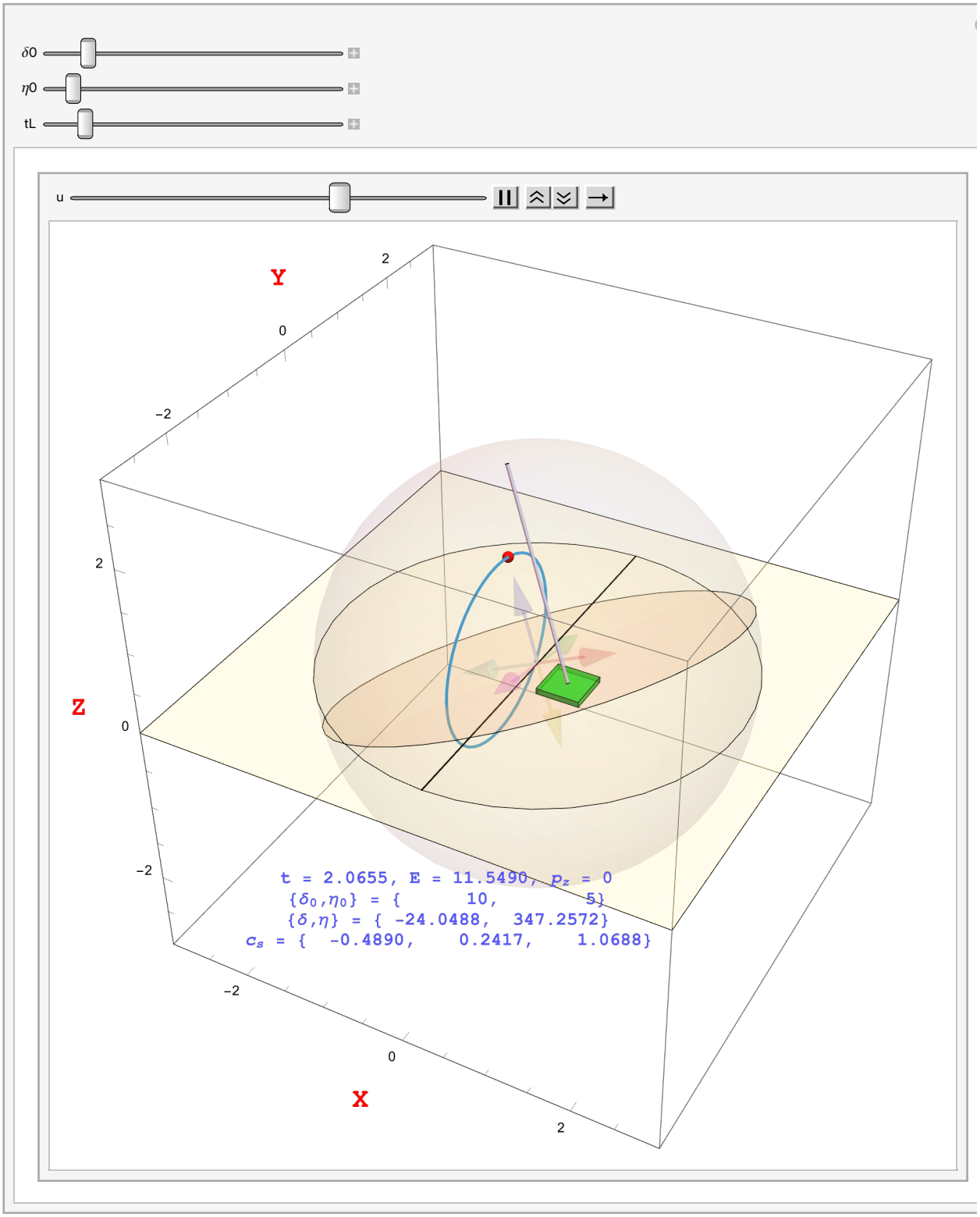
- *In another notebook,* How to Land a Rocket, *I show how to balance the baton via control theory at the unstable equilibrium, standing straight up.*

You can make the solution fail by moving the δ_0 slider all the way to the right to $\delta = (\pi/2) = 90°$. The η'' equation contains `Sec[δ[t]]`, which equals `ComplexInfinity` at $\delta = 90°$. That's the realization of ***gimbal lock***, which is inevitable in any attempt to cover the sphere with a single, global set of Euler angles.

In[35]:=
```
ClearAll[nfm, vfm, font, myFont];
myFont[color_, size_ : 18] :=
  Style[#, color, Bold, size, FontFamily → "Courier New"] &;
nfm[num_, dig_ : 10, dec_ : 4] := ToString[NumberForm[Chop@num, {dig, dec}]];
vfm[vec_] := ToString[
   NumberForm[#, {7, 4}, NumberSigns → {"-", " "}, NumberPadding → {" ", "0"}] & /@
    Chop@vec];
font = myFont[Lighter[Blue], 12];
With[{epsilon = 0.0001, h = 1 / 15., w = 1 / 4., vu = 3.},
 {ztxt = -1.5, xtxt = 0, ytxt = -2, znl = 0.3},
 {kart = Cuboid[{-w, -w, epsilon}, {w, w, epsilon + h}],
  floor = Polygon[{{-vu, -vu, 0}, {vu, -vu, 0}, {vu, vu, 0}, {-vu, vu, 0}}],
  axisLabelStyle = text ↦ Style[text, Red, Bold, 18, FontFamily → "Courier New"],
  cb = rig["cb"], thin = {0, 0, .0001}, s = 2, o = {0, 0, 0}},
 {c3 = Cylinder[{o, thin}, s cb[[3]]],
  b3 = Sphere[o, s cb[[3]]],
  t3x = Tube[{-s cb[[3]] e2, s cb[[3]] e2}, .01]},
 Manipulate[
  DynamicModule[
   {ics = {δ[0] == δ0, η[0] == η0, δ'[0] == 0, η'[0] == 0}, δx, Dδx, ηx, Dηx, stSolns},
   stSolns = NDSolve[Append[stEqns, ics] /. {Subscript[c, z] → 1.2},
     {δ[t], η[t], δ'[t], η'[t]}, {t, 0, tLim}];
   δx = stSolns[[1, 1, 2]]; Dδx = stSolns[[1, 3, 2]];
   ηx = stSolns[[1, 2, 2]]; Dηx = stSolns[[1, 4, 2]];
   Animate[
    Module[{renderPos = o, cs, rotfn,
      δ = δx /. t → u, η = ηx /. t → u, Dδ = Dδx /. t → u, Dη = Dηx /. t → u},
     cs = RotationMatrix[η, e1].RotationMatrix[δ, e2].cb;
     renderPos = -cs[[1]] e1 - cs[[2]] e2;
     rotfn = RotationTransform[η, e1]@*RotationTransform[δ, e2];
     Show[{Graphics3D[{
         Text[font["t = " <> nfm@u <>
            ", E = " <> nfm@(Tnum[{δ, η}, {Dδ, Dη}] + Vnum[{δ, η}]) <>
            ", p_z = " <> ToString@
```

```
            FortranForm@Chop[(Dδ Sin[η] - Dη Sin[δ] Cos[δ] Cos[η]), 10^-6]],
          {xtxt, ytxt, ztxt}],
         Text[font["{δ0,η0} = " <> vfm@{δ0 / °, η0 / °}], {xtxt, ytxt, ztxt - znl}],
         Text[font["{δ,η} = " <> vfm@{δ / °, η / °}], {xtxt, ytxt, ztxt - 2 znl}],
         Text[font["cs = " <> vfm@cs], {xtxt, ytxt, ztxt - 3 znl}],
         {Black, t3x},
         {White, Opacity[.7 / 8], c3, b3,
          GeometricTransformation[c3, rotfn]},
         GeometricTransformation[jack[0], rotfn],
         {Red, Sphere[cs, 1 / 16.]},
         {Yellow, Opacity[.3 / 4], floor},
         {Green, Opacity[.6], Translate[kart, renderPos]},
         {White, Opacity[.75],
          Translate[Translate[
            GeometricTransformation[rig["graphics primitives"], rotfn],
            cs], renderPos + (epsilon + h) e3]}},
        ImageSize → Large, Axes → True,
        AxesLabel → axisLabelStyle /@ {"X", "Y", "Z"},
        PlotRange → {{-vu, vu}, {-vu, vu}, {-vu, vu}}],
       ParametricPlot3D[RotationMatrix[ha, e1].RotationMatrix[δ, e2].cb,
        {ha, 0, 2 π}]}]],
   {u, 0, tL}, AnimationRate → .5]],
{{δ0, 10 °}, 0, 90 °},
{{η0, 5 °}, 0, 90 °},
{{tL, tLim / 10}, 0, tLim}]]
```

Out[40]=


δ0
η0
tL
u
Y
X
Z
2
0
−2
t = 2.0655, E = 11.5490, p_z = 0
{δ_0,η_0} = { 10, 5}
{δ,η} = { -24.0488, 347.2572}
c_s = { -0.4890, 0.2417, 1.0688}

- *The z component of the angular momentum is conserved because there is no gravitational torque in the z direction. A routine calculation shows the z component of angular momentum to be proportional to* $\left(x\dot{y} - y\dot{x}\right)$ *with* $x = c_z \sin\delta$, $y = c_z \cos\delta \sin\eta$.

In[41]:=
```
Module[{x, y, z, r, p},
  x[t] = Sin[δ[t]];
  y[t] = Cos[δ[t]] Sin[η[t]];
  z[t] = Cos[δ[t]] Cos[η[t]];
  r[t] = {x[t], y[t], z[t]};
  p[t] = D[r[t], t];
  (* fetch Z component at index 3 *)
  (r[t] × p[t])[[3]] // FullSimplify] // Echo[#, "p_z conserved :"] &;
```

» p_z conserved : `-Sin[η[t]] δ'[t] + Cos[δ[t]] Cos[η[t]] Sin[δ[t]] η'[t]`

Geometric Equations of Motion

Geodesic Equations

While standard Lagrangian mechanics produces equations of motion from scalar energy functionals, we can alternatively derive the equations of motion directly as geodesic equations in the geometry of the configuration space. One immediate benefit is that the geodesic equations yield directly intuitive terms:

$$\ddot{x}^{\mu} + \Gamma^{\mu}_{\nu\lambda}\dot{x}^{\nu}\dot{x}^{\lambda} = F^{\mu} \tag{1}$$

Coordinate accelerations $\ddot{x}^{\mu}$ + inertial accelerations $\Gamma^{\mu}_{\nu\lambda}\dot{x}^{\nu}\dot{x}^{\lambda}$ = forces from potentials F^{μ}.

The left-hand side of Equation 1 is a contraction of coordinate velocity $\dot{x}^{\nu}$ with covariant derivative of the velocity, which has two free indices, μ and ν:

$$\nabla_{\nu}\dot{x}^{\mu} \equiv \frac{\partial \dot{x}^{\mu}}{\partial x^{\nu}} + \Gamma^{\mu}_{\nu\lambda}\dot{x}^{\lambda} \tag{2}$$

Note, for later, that $\dot{x}^{\mu}$ here means a field of tangent vectors, one at at each x^{ν}. Later we will use the same symbol to denote the time derivative of a particular function of time, x^{μ}.

Absolute Derivative

Further define contraction with $\dot{x}^{\nu}$ as the ***Absolute Derivative*** D/dt :

$$\frac{D\dot{x}^{\mu}}{dt} \equiv \dot{x}^{\nu}\nabla_{\nu}\dot{x}^{\mu} = \frac{\partial x^{\nu}}{\partial t}\frac{\partial \dot{x}^{\mu}}{\partial x^{\nu}} + \Gamma^{\mu}_{\nu\lambda}\dot{x}^{\nu}\dot{x}^{\lambda} \tag{3}$$

By the chain rule

$$\frac{\partial x^{\nu}}{\partial t}\frac{\partial \dot{x}^{\mu}}{\partial x^{\nu}} = \frac{\partial \dot{x}^{\mu}}{\partial t} = \ddot{x}^{\mu} \tag{4}$$

we get concise equations of motion

$$\frac{D\dot{x}^{\mu}}{dt} = \ddot{x}^{\mu} + \Gamma^{\mu}_{\nu\lambda}\dot{x}^{\nu}\dot{x}^{\lambda} = F^{\mu} \tag{5}$$

Inertial Forces from the Connection

By projecting (contracting) the covariant-derivative-of-the-velocity on the velocity, we get the coordinate accelerations—we might have expected that—plus all the inertial forces from the connection Γ. Here is the interpretation of the inertial forces.

Out[ ]=

Symbol	Value	Cause ($v^{\mu}v^{\nu}$)	Effect (Force Direction)	Geometrical Interpretation
$\Gamma^{\delta}_{\eta\eta}$	$\neq 0$	Spinning in η	Push in δ	Centrifugal Force: "Spinning makes you fly outward."
$\Gamma^{\eta}_{\delta\eta}$	$\neq 0$	Mixing δ and η	Push in η	Coriolis Force: "Dipping while spinning swerves you sideways."
$\Gamma^{\eta}_{\delta\delta}$	0	Moving in δ	Push in η	Path Stability: "Walking pure North doesn't drift you East."
$\Gamma^{\delta}_{\delta\eta}$	0	Mixing δ and η	Push in δ	Force Orthogonality: "Coriolis twist is purely sideways."

These four components are a lot easier to intuit than the 32 independent components of the Christoffel symbols in 4D spacetime, which we have seen in earlier notebooks on general relativity. However, the geometrical meaning is 100% analogous. We get centrifugal and Coriolis forces if we move around curved, *SO*(3) coordinate space along geodesics in 3D. We get tidal forces and frame-dragging forces if we move around curved spacetime caused by massive objects in 4D.

All Forces from Connections

We have repeatedly claimed in various notebooks that ALL forces come from connections. What about those “forces from potentials” on the right-hand side of the new equations of motion? It turns out that they are the *time-time* components of 4D connections. We’ll leave that analysis for another notebook, and just take them on the right-hand side for now. That keeps the analysis smaller so we can explain it

intuitively.

Geometric Equations of Motion

Let's re-derive the equations of motion geometrically, then show them equal to our old equations of motion, derived via the Euler-Lagrange process. We'll repeat the new derivation twice using two different sectors of the $\mathcal{UD}$ toolkit: first, the newer, component-based "indexer" sector and then the older, rule-based "$\mathcal{UD}$" sector. These sectors can be freely mixed and matched. The indexer sector has the advantage of conciseness, but at the cost of explicit index up-ness and down-ness. The indexer sector is analogous to a weakly typed sub-language. The $\mathcal{UD}$ sector requires more code, but keeps track of the variance and valence. The $\mathcal{UD}$ sector is analogous to a strongly typed sub-language.

Start with the same old kinetic and potential energies, geometrically, this time:

Kinetic Energy

Fish kinetic energy out of the definitions above, knowing that $m \to 1$ and $c_z \to 1.2$ have already been applied. Or just intuit kinetic energy from the familiar $\frac{1}{2} I_1 \omega^2$. Or just read the old notebook. In any event:

```
T = 1 / 2 (1.2) ^ 2 (Dδ ^ 2 + (1 / 2 (1 + Cos[2 δ])) Dη ^ 2)
```

is the code. In traditional notation, it is

$$T = \frac{1}{2} m c_z^2 \left((\delta')^2 + (\eta')^2 \left(\frac{1}{2} (1 + \cos 2\,\delta) \right) \right) = \frac{1}{2} m c_z^2 \left((\delta')^2 + (\eta')^2 \cos^2 \delta \right) \tag{6}$$

via the half-angle formula $\left(\cos^2 \delta = \frac{1}{2} (1 + \cos 2\,\delta) \right)$. Of course, δ, η, and T are functions of coordinate time, t, here Newtonian time.

- *c_z is half the length of the baton, the height of the center of mass in the Z direction when the baton is straight up. Nothing to do with c, the speed of light.*

Metric Tensor times Velocities

It's obvious that T has syntactic structure as a row vector of coordinate velocities times a matrix times a column vector of coordinate velocities:

$$T = \frac{1}{2} m c_z^2 \begin{pmatrix} \delta' & \eta' \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 \\ 0 & \cos^2 \delta \end{pmatrix} \cdot \begin{pmatrix} \delta' \\ \eta' \end{pmatrix} \tag{7}$$

See this in tensorial terms as contraction of a covariant velocity covector (index lowered by contraction against a metric tensor, $g_{\mu\nu}$) with a contravariant velocity vector:

$$\frac{1}{2} m c_z^2 \, g_{\mu\nu} \dot{x}^\mu \dot{x}^\nu = \frac{1}{2} m c_z^2 \, \dot{x}_\nu \dot{x}^\nu \tag{8}$$

Except for the factor of $\left(m c_z^2 / 2 \right)$, this is square length of the tangent to a line element, metricized by $g_{\mu\nu}$, along a curve specified by $x(t)$. Configuration space $\{\delta, \eta\}$ is not flat, it's a sphere where separation between lines of longitude shrink with $\cos \delta$. That's the coordinate realization of the topological fact that $SO(3)$ has an essential singularity.

Reproduce Equation 6

First, reproduce Equation 6 in the indexer sector of $\mathcal{UD}$ via the functions `MakeIndexer` and `ContractIndexedAll`. Then move on to the equations of motion via `ChristoffelsFromMetricIndexer`.

Set up coordinates and a time-injector. We'll need the time-injector because explicit time dependence, e.g., $\delta[t]$, $\delta'[t]$, etc., is necessary for numerical solution via **`NDSolve`**.

In[42]:=
```
coords = {δ, η};
ClearAll[injectTime];
injectTime =
  Echo[Flatten[(c ↦ {c → c[t], c' → c'[t]}) /@ coords], "time-injection rules :"];
```

» time–injection rules : $\{\delta \to \delta[t],\ \delta' \to \delta'[t],\ \eta \to \eta[t],\ \eta' \to \eta'[t]\}$

In[45]:=
```
With[{g = m cz^2 ({{1, 0}, {0, Cos[δ]^2}})}, metric = MakeIndexer[g, coords]];
Echo[Table[metric[μ, ν] /. injectTime,
    {μ, coords}, {ν, coords}] // MatrixForm, "metric g_μν ="];

ClearAll[vels, accels, v, a];
vels = (c ↦ c'[t]) /@ coords;
accels = (c ↦ c''[t]) /@ coords;
v = MakeIndexer[vels, coords];
a = MakeIndexer[accels, coords];
Echo[Table[v[κ], {κ, coords}], "coordinate velocity ẋ ="];

Echo[ContractIndexedAll[
   (* looks just like the school formula 1/2 mv^2 *)
   {μ, ν} ↦ 1/2 (metric[μ, ν] /. injectTime) * v[μ] * v[ν],
   coords], "Kinetic Energy T ="];
```

» metric $g_{\mu\nu}$ = $\begin{pmatrix} cz^2\, m & 0 \\ 0 & cz^2\, m\, \text{Cos}[\delta[t]]^2 \end{pmatrix}$

» coordinate velocity ẋ = $\{\delta'[t],\ \eta'[t]\}$

» Kinetic Energy T = $\frac{1}{2}\, cz^2\, m\, \delta'[t]^2 + \frac{1}{2}\, cz^2\, m\, \text{Cos}[\delta[t]]^2\, \eta'[t]^2$

reproducing Equation 6.

QED■

Connection $\Gamma^{\mu}_{\nu\lambda}$

We'll need the connection, Γ, computed from the metric and its inverse.

In[54]:=
```
With[{ig = Inverse[m cz^2 {{1, 0}, {0, Cos[δ]^2}}]},
  invMetric = MakeIndexer[ig, coords];];
Echo[Table[invMetric[μ, ν] /. injectTime,
    {μ, coords}, {ν, coords}] // MatrixForm, "g^μν (inverse of g_μν) ="];

ΓPendulum[μ_, ν_, λ_] :=
  ChristoffelsFromMetricIndexer[metric, invMetric, μ, ν, λ, coords];
Echo[Table[ΓPendulum[μ, ν, λ] /. injectTime,
    {μ, coords}, {ν, coords}, {λ, coords}] // MatrixForm, "Γ^μ_νλ ="];
```

» $g^{\mu\nu}$ (inverse of $g_{\mu\nu}$) = $\begin{pmatrix} \frac{1}{cz^2\, m} & 0 \\ 0 & \frac{\text{Sec}[\delta[t]]^2}{cz^2\, m} \end{pmatrix}$

» $\Gamma^{\mu}{}_{\nu\lambda}$ = $\begin{pmatrix} \begin{pmatrix} 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ \text{Cos}[\delta[t]]\ \text{Sin}[\delta[t]] \end{pmatrix} \\ \begin{pmatrix} 0 \\ -\text{Tan}[\delta[t]] \end{pmatrix} & \begin{pmatrix} -\text{Tan}[\delta[t]] \\ 0 \end{pmatrix} \end{pmatrix}$

- *Note the presence of* `Sec[δ[t]]` *in the* `[[2,2]]` *element of the metric. The metric diverges when* `δ[t]→ 90°`.

Curvature Forces $\Gamma^{\mu}_{\nu\lambda}\, \dot{x}^{\nu}\, \dot{x}^{\lambda}$

New function for the toolkit v1.13.0.

This function internally produces a time injector rather than relying on an external definition. Its output is the full contravariant vector (up index) of inertial forces.

In[58]:=
```
ClearAll[CurvatureForcesFromConnectionIndexer];
CurvatureForcesFromConnectionIndexer[v_, Γ_, time_, coords_] :=
  With[{injectTime = Flatten[(c ↦ {c → c[time], c' → c'[time]}) /@ coords]},
   Table[
    ContractIndexedAll[
     {ν, λ} ↦ (Γ[μ, ν, λ] /. injectTime) v[ν] × v[λ],
     coords],
    {μ, coords}]];

Echo[CurvatureForcesFromConnectionIndexer[v, ΓPendulum, t, coords] //
   MatrixForm, "Γ^μ_νλ ẋ^ν ẋ^λ :"];
```

» $\Gamma^{\mu}_{\nu\lambda}\, \dot{x}^{\nu}\, \dot{x}^{\lambda}$: $\begin{pmatrix} \text{Cos}[\delta[t]]\ \text{Sin}[\delta[t]]\ \eta'[t]^2 \\ -2\ \text{Tan}[\delta[t]]\ \delta'[t]\ \eta'[t] \end{pmatrix}$

Absolute Derivative $\frac{D\dot{x}^{\mu}}{dt} \equiv \dot{x}^{\nu} \nabla_{\nu} \dot{x}^{\mu} = \ddot{x}^{\mu} + \Gamma^{\mu}_{\nu\lambda}\, \dot{x}^{\nu}\, \dot{x}^{\lambda}$

New function for the toolkit v1.13.0.

Adds an explicit coordinate acceleration term to the inertial forces and produces the entire vector.

- *We have to do it this way because {δ'[t], η'[t]} is not , here, a field of velocity vectors, one for each point {δ, η}, but rather a particular velocity that depends on time. Therefore, for instance, the derivative* $\left(\partial \dot{\delta} \middle/ \partial \delta\right)$ *or* `D[v[t],coord]`*, doesn't make sense here. We can't mechanically compute the chain rule of Equation 4. Wolfram reduces* `D[v[t],coord]` *to zero. This is not whimsical, but rather a consequence of the fact that we've already assumed a function of time and we're no longer tracking the field.*

In[61]:=
```
ClearAll[AbsoluteDerivativeFromConnectionIndexer];
AbsoluteDerivativeFromConnectionIndexer[v_, Γ_, time_, coords_] :=
  With[{
    inertialTerms = CurvatureForcesFromConnectionIndexer[v, Γ, time, coords],
    coordinateAcceleration = (c ↦ c''[time]) /@ coords},
   inertialTerms + coordinateAcceleration];

Echo[AbsoluteDerivativeFromConnectionIndexer[v, ΓPendulum, t, coords] //
   MatrixForm, "x⃜^μ + Γ^μ_νλ ẋ^ν ẋ^λ :"];
```

» $\ddot{x}^{\mu} + \Gamma^{\mu}_{\nu\lambda}\, \dot{x}^{\nu}\, \dot{x}^{\lambda}$:
$$\begin{pmatrix} \text{Cos}[\delta[t]]\ \text{Sin}[\delta[t]]\ \eta'[t]^2 + \delta''[t] \\ -2\ \text{Tan}[\delta[t]]\ \delta'[t]\ \eta'[t] + \eta''[t] \end{pmatrix}$$

Potential Scalar Field

New function for the toolkit v1.13.0.

Gravitational force is the negative gradient of the potential.

In[64]:=
```
ClearAll[V];
Echo[V = m g cz Cos[δ] Cos[η], "V =="];
```

» V = 9.81 cz m Cos[δ] Cos[η]

$F^{\mu} = -g^{\mu\nu}\,\partial_{\nu} V$

$F^{\mu} = -g^{\mu\nu}\,\partial_{\nu}(g\, c_z\, m \cos\delta \cos\eta)$

In[66]:=
```
ClearAll[PotentialScalarFieldFromMetricIndexer];
PotentialScalarFieldFromMetricIndexer[V_, invMetric_, time_, coords_] :=
  With[{injectTime = Flatten[(c ↦ {c → c[time], c' → c'[time]}) /@ coords]},
   Table[
     -ContractIndexedAll[
       {ν} ↦ invMetric[μ, ν] * D[V, ν],
       coords],
     {μ, coords}] /. injectTime];

Echo[PotentialScalarFieldFromMetricIndexer[V, invMetric, t, coords] //
    FullSimplify // MatrixForm, "F^μ=-g^μ^ν∂_νV :"];
```

» $F^{\mu}=-g^{\mu\nu}\partial_{\nu}V$:
$$\begin{pmatrix} \frac{9.81\,\text{Cos}[\eta[t]]\,\text{Sin}[\delta[t]]}{cz} \\ \frac{9.81\,\text{Sec}[\delta[t]]\,\text{Sin}[\eta[t]]}{cz} \end{pmatrix}$$

Equations of Motion

New Equations

Everything is in component-indexed form.

In[69]:=
```
nEqns = MapThread[
    Equal,
    {AbsoluteDerivativeFromConnectionIndexer[v, ΓPendulum, t, coords],
     PotentialScalarFieldFromMetricIndexer[V, invMetric, t, coords]}];
(nStateSpaceForm = Solve[nEqns, accels] // FullSimplify // Flatten) // MatrixForm
```

Out[70]//MatrixForm=
$$\begin{pmatrix} \delta''[t] \to \frac{\text{Sin}[\delta[t]]\,\left(9.81\,\text{Cos}[\eta[t]]-1.\,cz\,\text{Cos}[\delta[t]]\,\eta'[t]^2\right)}{cz} \\ \eta''[t] \to \frac{9.81\,\text{Sec}[\delta[t]]\,\text{Sin}[\eta[t]]}{cz} + 2.\,\text{Tan}[\delta[t]]\,\delta'[t]\,\eta'[t] \end{pmatrix}$$

Old Equations

From the old notebook, replacing c_z with `cz` to compensate for a historical accident of presentation:

In[71]:=
```
oEqns = stEqns /. {c_z → cz};
Echo[
  (oStateSpaceForm = Solve[oEqns, accels] // FullSimplify // Flatten) // MatrixForm,
  "old state-space eqns :"];
```

» old state-space eqns :
$$\begin{pmatrix} \delta''[t] \to \frac{\text{Sin}[\delta[t]]\,\left(9.81\,\text{Cos}[\eta[t]]-1.\,cz\,\text{Cos}[\delta[t]]\,\eta'[t]^2\right)}{cz} \\ \eta''[t] \to \frac{9.81\,\text{Sec}[\delta[t]]\,\text{Sin}[\eta[t]]}{cz} + 2.\,\text{Tan}[\delta[t]]\,\delta'[t]\,\eta'[t] \end{pmatrix}$$

Old == New

In[73]:=
```
Echo[(nStateSpaceForm /. Rule → Equal /. oStateSpaceForm) // FullSimplify,
  "old == new :"];
```

» old == new : {True, True}

QED■

In the $\mathcal{UD}$ Sector

In the above computations in the indexer sector, there is one nagging programming nit. We computed the gradient of the potential through an explicit contraction in **PotentialScalarField**, rather than via $\mathcal{UD}$'s gradient operator, **GradRaised**. To put **GradRaised** to work, we must redo the entire computation in the $\mathcal{UD}$ sector, where **GradRaised** was developed. We'll find the code more verbose, but producing the same result.

In[74]:=
```
ClearAll[gDD, gUU, gDDMat, gDDrules, gUUrules];
Echo[(gDDMat = m cz^2 ({{1, 0}, {0, Cos[δ]^2}})) // MatrixForm, "g_μν (matrix) :"];
```

» $g_{\mu\nu}$ (matrix) : $\begin{pmatrix} cz^2\, m & 0 \\ 0 & cz^2\, m\, \text{Cos}[\delta]^2 \end{pmatrix}$

Recall how to convert a matrix into $\mathcal{UD}$ rules:

In[76]:=
```
?? MatrixToUDRules
```

Out[76]=

Symbol

Global`MatrixToUDRules

Definitions

```
MatrixToUDRules[mat_, g_Symbol, UorD_ /; UorD === 𝒰 || UorD === 𝒟, coords_List] :=
Module[{dim, rules}, dim = Length[mat];
rules = Flatten[
Table[g[UorD[coords[[i]]], UorD[coords[[j]]]] → mat[[i, j]], {i, 1, dim}, {j, 1, dim}]];
Select[rules, #1[[2]] =!= 0 &]]
```

Full Name Global`MatrixToUDRules

This is a sparse-matrix representation, alternative but equivalent to the full matrices above.

In[77]:=
```
Echo[gDDrules = MatrixToUDRules[gDDMat, gDD, 𝒟, coords], "g_μν rules :"];
Echo[gUUrules = MatrixToUDRules[gDDMat // Inverse, gUU, 𝒰, coords], "g^μν rules :"];
offDiagonalRules = {
    gUU[𝒰[a_], 𝒰[b_]] /; a =!= b → 0,
    gDD[𝒟[a_], 𝒟[b_]] /; a =!= b → 0};
```

» $g_{\mu\nu}$ rules : $\left\{\text{gDD}_{\delta\,\delta} \to \text{cz}^2\,\text{m},\ \text{gDD}_{\eta\,\eta} \to \text{cz}^2\,\text{m}\,\text{Cos}[\delta]^2\right\}$

» $g^{\mu\nu}$ rules : $\left\{\text{gUU}^{\delta\,\delta} \to \frac{1}{\text{cz}^2\,\text{m}},\ \text{gUU}^{\eta\,\eta} \to \frac{\text{Sec}[\delta]^2}{\text{cz}^2\,\text{m}}\right\}$

Absolute DerivativeUD

Recall how to compute Γ components in the $\mathcal{UD}$ sector: via rules

In[80]:=
```
?? CalculateGammaComponent
```

Out[80]=

Symbol

Global`CalculateGammaComponent

Definitions

```
CalculateGammaComponent[σ_, μ_, ν_, gDD_Symbol, gDDRules_,
gInvUU_Symbol, gInvUURules_, coords_] := EvaluateUDPartials[
ContractAll[ChristoffelsFromMetric[gDD, gInvUU, σ, μ, ν], coords] /. gDDRules /.
gInvUURules]
```

Full Name Global`CalculateGammaComponent

CurvatureForcesUD

New functions for the toolkit v1.13.0.

In[81]:=
```
ClearAll[CurvatureForcesUD];
CurvatureForcesUD[
    gDD_Symbol, gDDRules_,
    gUU_Symbol, gUURules_,
    time_Symbol,
    coords_] :=
  With[{injectTime = Flatten[(c ↦ {c → c[time], c' → c'[time]}) /@ coords]},
   Table[
    ContractIndexedAll[
     {ν, λ} ↦ (CalculateGammaComponent[
           μ, ν, λ,
           gDD, gDDRules,
           gUU, gUURules,
           coords] /. injectTime) * D[ν[time], time] * D[λ[time], time],
     coords],
    {μ, coords}]];
```

In[83]:=
```
ClearAll[AbsoluteDerivativeUD];
AbsoluteDerivativeUD[
    gDD_Symbol, gDDRules_, (* Metric symbol and rules *)
    gUU_Symbol, gUURules_,   (* Inverse Metric symbol and rules *)
    time_Symbol,      (* The time variable, e.g. t *)
    coords_   (* The list of coordinate symbols *)
  ] :=
  With[{
    inertialTerms = CurvatureForcesUD[gDD, gDDRules, gUU, gUURules, time, coords],
    coordinateAcceleration = (c ↦ c''[time]) /@ coords},
   inertialTerms + coordinateAcceleration];


Echo[AbsoluteDerivativeUD[
    gDD, gDDrules ~Join~ offDiagonalRules,
    gUU, gUUrules ~Join~ offDiagonalRules,
    t, coords] // MatrixForm, "\!\(\*OverDot[x]^\[Mu]\) + \!\(\*SubsuperscriptBox[\(\[CapitalGamma]\), \(\[Nu]\ \[Lambda]\), \(\[Mu]\)]\) \!\(\*OverDot[x]^\[Nu]\) \!\(\*OverDot[x]^\[Lambda]\) :"];
```

» $\ddot{x}^{\mu} + \Gamma^{\mu}_{\nu\lambda}\,\dot{x}^{\nu}\,\dot{x}^{\lambda}$:
$$\begin{pmatrix} \text{Cos}[\delta[t]]\ \text{Sin}[\delta[t]]\ \eta'[t]^2 + \delta''[t] \\ -2\ \text{Tan}[\delta[t]]\ \delta'[t]\ \eta'[t] + \eta''[t] \end{pmatrix}$$

PotentialScalarFieldUD

New function for the toolkit v1.13.0.

Here's were we use `GradRaised`.

In[86]:=
```
ClearAll[PotentialScalarFieldUD];
PotentialScalarFieldUD[V_, gUU_Symbol, gUURules_, time_Symbol, coords_] :=
  With[{injectTime = Flatten[(c ↦ {c → c[time], c' → c'[time]}) /@ coords],
    GradV = GradRaised[V, gUU, gUURules, coords]},
   -GradV /. injectTime
  ];

Echo[PotentialScalarFieldUD[V, gUU, gUUrules ~ Join ~ offDiagonalRules, t, coords] //
    FullSimplify // MatrixForm, "-∇V :"];
```

» −∇V :
$$\begin{pmatrix} \frac{9.81\,\text{Cos}[\eta[t]]\,\text{Sin}[\delta[t]]}{cz} \\ \frac{9.81\,\text{Sec}[\delta[t]]\,\text{Sin}[\eta[t]]}{cz} \end{pmatrix}$$

The Final Test

In[89]:=
```
nEqnsUD = MapThread[
    Equal,
    {AbsoluteDerivativeUD[
      gDD, gDDrules ~ Join ~ offDiagonalRules,
      gUU, gUUrules ~ Join ~ offDiagonalRules,
      t, coords],
     PotentialScalarFieldUD[
      V,
      gUU, gUUrules ~ Join ~ offDiagonalRules,
      t, coords]}] /. {g → 9.81};
Echo[(nStateSpaceFormUD = Solve[nEqnsUD, accels] // FullSimplify // Flatten) //
   MatrixForm,
  "new state-space eqns in 𝒰𝒟 :"];
Echo[(nStateSpaceFormUD /. Rule → Equal /. oStateSpaceForm) // FullSimplify,
  "old == new eqns in 𝒰𝒟 :"];
```

» new state–space eqns in $\mathcal{UD}$:
$$\begin{pmatrix} \delta''[t] \to \frac{\text{Sin}[\delta[t]]\,\left(9.81\,\text{Cos}[\eta[t]] - 1.\,cz\,\text{Cos}[\delta[t]]\,\eta'[t]^2\right)}{cz} \\ \eta''[t] \to \frac{9.81\,\text{Sec}[\delta[t]]\,\text{Sin}[\eta[t]]}{cz} + 2.\,\text{Tan}[\delta[t]]\,\delta'[t]\,\eta'[t] \end{pmatrix}$$

» old == new eqns in $\mathcal{UD}$: {True, True}

QED■

SO(3)

Up to now, we have entertained only two angles, δ and η. That means our metric and connections do not span the full $SO(3)$ group of three angles, but inhabit a quotient group, $SO(3)/SO(2)$. One angle, ψ, lives alone and ignored in $SO(2)$ as $\begin{pmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{pmatrix}$, a matrix whose two dimensions give the "2" to

"$SO(2)$."

What is ψ? The only thing it could be: twist around the long axis of the baton, changing the baton from a pendulum that falls over into a spinning, cylindrically symmetric top that gyroscopically resists falling over.

Gyroscopic forces are again inertial, arising from a new metric and connection, but we get them with no further manual effort. Just turn the crank and get a spinning symmetrical top.

Kinetic Energy

Let I_3 be the moment of inertia around the ψ axis. Set I_3 to be some manipulable multiple of $I_1 = mc_z^2$, the δ-η moment of inertia about the base of the baton.

The η-ψ cross-terms in the kinetic energy arise from projecting η-rotation against the ψ axis. As the baton starts to fall, its spinning inertia about the ψ axis flings it back up, decreasing η and causing precession and nutation.

To consider the extremes:

- when $\delta = 0$, the baton is straight up and and η-rotation has no projection on the ψ-spin axis.
- When $\delta = 90°$, the baton lies entirely in the equatorial plane and all η rotation is around the ψ axis.

The projection of the η-rotation against the ψ axis generates the gyroscopic forces, and we'll see a term corresponding to that projection in the connection.

Gauge Potential and Gauge Connection

The off-diagonal element is the $SO(3)$ ***gauge potential***, analogous to A_μ in electrodynamics mentioned in an earlier notebook, *Gauge Theory in* $\mathcal{UD}$. In a flat, uncoupled configuration manifold with no cross-term, cone-rolling the baton (η) and spinning the baton (ψ) would be independent. However, because of the geometry of $SO(3)$, moving the configuration along an η-coordinate line leaks into the ψ coordinate and vice versa. The amount of leak is exactly $I_3 \sin\delta$. The geometric leak is the ***gauge connection***, and its covariant derivative generates the Coriolis forces that physically manifest as gyroscopic precession.

Topology

The fundamental reason for this leak is topological. The total configuration space, $SO(3)$, is a non-trivial fibre bundle (see references). It is not merely a flat 2D sphere $\{\delta, \eta\}$ with a 1D circle (ψ) glued on via Cartesian product. That would be a *trivial* fibre bundle.

As the baton moves around the sphere, the spin fibers suffer a global topological twist. Gyroscopic precession and nutation are the physical manifestation of the baton's navigating this twisted space.

Now, just mimic the indexer-sector development above. Precession and nutation arise spontaneously.

In[92]:=
```
ClearAll[coords3, vels3, accels3, v3, a3, Γ3, I3, I1, cz, m];
coords3 = {δ, η, ψ};
metric3 = {{I1, 0, 0}, {0, (I1 Cos[δ]^2 + I3 Sin[δ]^2), I3 Sin[δ]}, {0, I3 Sin[δ], I3}};
g3 = MakeIndexer[metric3, coords3];
ig3 = MakeIndexer[Inverse@metric3, coords3];
Echo[Table[g3[μ, ν], {μ, coords3}, {ν, coords3}] // MatrixForm, "g3_μν :"];
Echo[Table[ig3[μ, ν], {μ, coords3}, {ν, coords3}] // MatrixForm, "g3^μν :"];
```

» $g3_{\mu\nu}$:
$$\begin{pmatrix} I1 & 0 & 0 \\ 0 & I1\,\text{Cos}[\delta]^2 + I3\,\text{Sin}[\delta]^2 & I3\,\text{Sin}[\delta] \\ 0 & I3\,\text{Sin}[\delta] & I3 \end{pmatrix}$$

» $g3^{\mu\nu}$:
$$\begin{pmatrix} \frac{1}{I1} & 0 & 0 \\ 0 & \frac{\text{Sec}[\delta]^2}{I1} & -\frac{\text{Sec}[\delta]\,\text{Tan}[\delta]}{I1} \\ 0 & -\frac{\text{Sec}[\delta]\,\text{Tan}[\delta]}{I1} & \frac{\text{Sec}[\delta]^2\,(I1^2\,\text{Cos}[\delta]^2 + I1\,I3\,\text{Sin}[\delta]^2)}{I1^2\,I3} \end{pmatrix}$$

In[99]:=
```
Echo[vels3 = (c ↦ c'[t]) /@ coords3, "velocities :"];
v3 = MakeIndexer[vels3, coords3];
Echo[accels3 = (c ↦ c''[t]) /@ coords3, "accelerations :"];
a3 = MakeIndexer[accels3, coords3];
injectTime3 =
  Echo[Flatten[(c ↦ {c → c[t], c' → c'[t]}) /@ coords3], "time-injection :"];
```

» velocities : $\{\delta'[t], \eta'[t], \psi'[t]\}$

» accelerations : $\{\delta''[t], \eta''[t], \psi''[t]\}$

» time–injection : $\{\delta \to \delta[t], \delta' \to \delta'[t], \eta \to \eta[t], \eta' \to \eta'[t], \psi \to \psi[t], \psi' \to \psi'[t]\}$

For inspection; not used in the sequel.

In[104]:=
```
Echo[ContractIndexedAll[
  {μ, ν} ↦ 1/2 (g3[μ, ν] /. injectTime3) * v3[μ] * v3[ν],
  coords3] // FullSimplify, "Kinetic Energy T :"];
```

» Kinetic Energy T :

$$\frac{1}{2}\left(I1\,\delta'[t]^2 + \left(I1\,\text{Cos}[\delta[t]]^2 + I3\,\text{Sin}[\delta[t]]^2\right)\eta'[t]^2 + 2\,I3\,\text{Sin}[\delta[t]]\,\eta'[t]\,\psi'[t] + I3\,\psi'[t]^2\right)$$

Connection and Equations of Motion

In[105]:=
```
Γ3[μ_, ν_, λ_] := ChristoffelsFromMetricIndexer[g3, ig3, μ, ν, λ, coords3];
Echo[Table[Γ3[μ, ν, λ], {μ, coords3}, {ν, coords3}, {λ, coords3}] // FullSimplify //
    MatrixForm, "Γ3^μ_νλ :"];
Echo[(ad3 = AbsoluteDerivativeFromConnectionIndexer[v3, Γ3, t, coords3] //
       FullSimplify) // MatrixForm, "inertial forces :"];
Echo[
  (pf3 = PotentialScalarFieldFromMetricIndexer[V, ig3, t, coords3] // FullSimplify) //
   MatrixForm, "potential forces :"];
Echo[(eqns3 = MapThread[Equal, {ad3, pf3}] // FullSimplify), "eqns3 :"];
```

» $\Gamma 3^{\mu}_{\nu\lambda}$:

$$\begin{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ \frac{(I1-I3)\,\cos[\delta]\,\sin[\delta]}{I1} \\ -\frac{I3\,\cos[\delta]}{2\,I1} \end{pmatrix} & \begin{pmatrix} 0 \\ -\frac{I3\,\cos[\delta]}{2\,I1} \\ 0 \end{pmatrix} \\ \begin{pmatrix} 0 \\ \frac{(-2\,I1+I3)\,\tan[\delta]}{2\,I1} \\ \frac{I3\,\sec[\delta]}{2\,I1} \end{pmatrix} & \begin{pmatrix} \frac{(-2\,I1+I3)\,\tan[\delta]}{2\,I1} \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} \frac{I3\,\sec[\delta]}{2\,I1} \\ 0 \\ 0 \end{pmatrix} \\ \begin{pmatrix} 0 \\ \frac{\cos[\delta]\,\left(I1+(2\,I1-I3)\,\tan[\delta]^2\right)}{2\,I1} \\ -\frac{I3\,\tan[\delta]}{2\,I1} \end{pmatrix} & \begin{pmatrix} \frac{\cos[\delta]\,\left(I1+(2\,I1-I3)\,\tan[\delta]^2\right)}{2\,I1} \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} -\frac{I3\,\tan[\delta]}{2\,I1} \\ 0 \\ 0 \end{pmatrix} \end{pmatrix}$$

» inertial forces :

$$\begin{pmatrix} \frac{(I1-I3)\,\cos[\delta[t]]\,\sin[\delta[t]]\,\eta'[t]^2-I3\,\cos[\delta[t]]\,\eta'[t]\,\psi'[t]+I1\,\delta''[t]}{I1} \\ \frac{\delta'[t]\,((-2\,I1+I3)\,\tan[\delta[t]]\,\eta'[t]+I3\,\sec[\delta[t]]\,\psi'[t])}{I1}+\eta''[t] \\ \frac{\delta'[t]\,((I1\,\cos[\delta[t]]+(2\,I1-I3)\,\sin[\delta[t]]\,\tan[\delta[t]])\,\eta'[t]-I3\,\tan[\delta[t]]\,\psi'[t])}{I1}+\psi''[t] \end{pmatrix}$$

» potential forces :

$$\begin{pmatrix} \frac{9.81\,cz\,m\,\cos[\eta[t]]\,\sin[\delta[t]]}{I1} \\ \frac{9.81\,cz\,m\,\sec[\delta[t]]\,\sin[\eta[t]]}{I1} \\ -\frac{9.81\,cz\,m\,\sin[\eta[t]]\,\tan[\delta[t]]}{I1} \end{pmatrix}$$

» eqns3 :

$$\left\{\frac{(I1-I3)\,\cos[\delta[t]]\,\sin[\delta[t]]\,\eta'[t]^2-I3\,\cos[\delta[t]]\,\eta'[t]\,\psi'[t]+I1\,\delta''[t]}{I1}==\frac{9.81\,cz\,m\,\cos[\eta[t]]\,\sin[\delta[t]]}{I1},\right.$$
$$\frac{\delta'[t]\,((-2\,I1+I3)\,\tan[\delta[t]]\,\eta'[t]+I3\,\sec[\delta[t]]\,\psi'[t])}{I1}+\eta''[t]==\frac{9.81\,cz\,m\,\sec[\delta[t]]\,\sin[\eta[t]]}{I1},$$
$$\left.\frac{\delta'[t]\,((I1\,\cos[\delta[t]]+(2\,I1-I3)\,\sin[\delta[t]]\,\tan[\delta[t]])\,\eta'[t]-I3\,\tan[\delta[t]]\,\psi'[t])}{I1}+\psi''[t]==-\frac{9.81\,cz\,m\,\sin[\eta[t]]\,\tan[\delta[t]]}{I1}\right\}$$

Check Equations against $SO(3)/SO(2)$

Show that the $SO(3)$ equations devolve to the known $SO(3)/SO(2)$ equations when $I3 \to 0$. Verify by inspection.

In[110]:=
```
devoEqns =
  (((Solve[eqns3 /. {I3 → 0, I1 → cz^2, m → 1}, accels3][[1]])[[1 ;; 2]]) //. Rule → Equal) //
   FullSimplify;
Echo[devoEqns /. oStateSpaceForm, "SO(3)/.(I3→0) == stEqns? :"];
```

» SO(3)/.(I3→0) == stEqns? : {True, True}

QED■

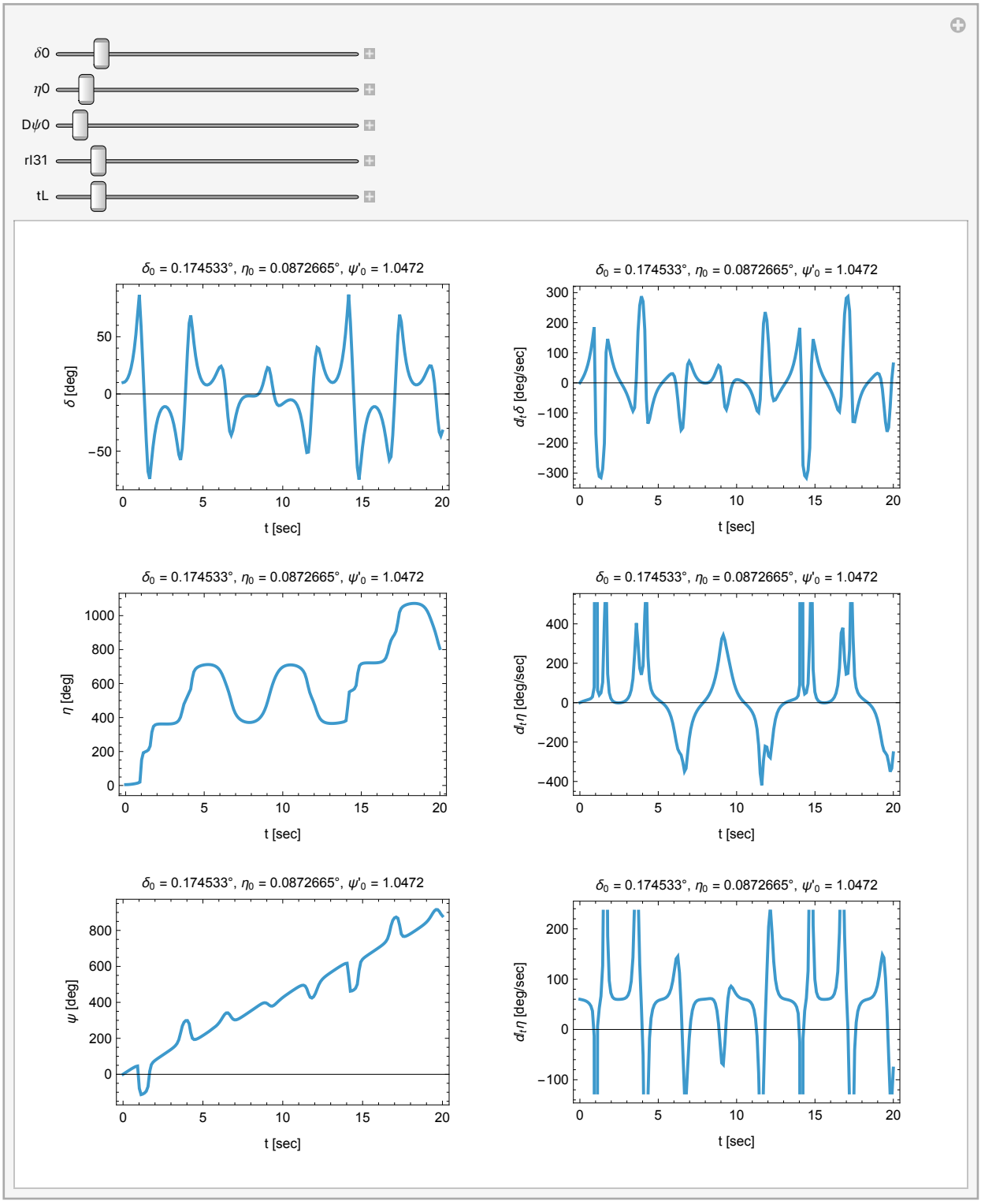
Plot the Solutions

The following plots are fascinating if not illuminating. They serve mostly as a plausibility check.

In[112]:=

```
radPerSec[rpm_] := 2. π / 60. * rpm;
Manipulate[DynamicModule[{
   ics = {δ[0] == δ0, η[0] == η0, ψ[0] == 0, δ'[0] == 0, η'[0] == 0, ψ'[0] == Dψ0},
   δx, ηx, ψx, Dδx, Dηx, Dψx, flt},
  soln3 = NDSolve[
    Append[eqns3, ics] //. {m → 1, cz → 1.2, I1 → 1.2^2, I3 → rI31 * I1},
    {δ[t], η[t], ψ[t], δ'[t], η'[t], ψ'[t]}, {t, 0, tLim}];
  δx = soln3[[1, 1, 2]]; Dδx = soln3[[1, 4, 2]];
  ηx = soln3[[1, 2, 2]]; Dηx = soln3[[1, 5, 2]];
  ψx = soln3[[1, 3, 2]]; Dψx = soln3[[1, 6, 2]];
  flt = "δ0 = " <> ToString[δ0] <>
    "°, η0 = " <> ToString[η0] <> "°, ψ'0 = " <> ToString[Dψ0];
  GraphicsGrid[{
    {Plot[δx / °, {t, 0, tL},
      FrameLabel → {{"δ [deg]", ""}, {"t [sec]", flt}}, Frame → True],
     Plot[Dδx / °, {t, 0, tL},
      FrameLabel → {{"dδ/dt [deg/sec]", ""}, {"t [sec]", flt}}, Frame → True]},
    {Plot[ηx / °, {t, 0, tL},
      FrameLabel → {{"η [deg]", ""}, {"t [sec]", flt}}, Frame → True],
     Plot[Dηx / °, {t, 0, tL},
      FrameLabel → {{"dη/dt [deg/sec]", ""}, {"t [sec]", flt}}, Frame → True]},
    {Plot[ψx / °, {t, 0, tL},
      FrameLabel → {{"ψ [deg]", ""}, {"t [sec]", flt}}, Frame → True],
     Plot[Dψx / °, {t, 0, tL},
      FrameLabel → {{"dη/dt [deg/sec]", ""}, {"t [sec]", flt}}, Frame → True]}
  }]],
 {{δ0, 10. °}, 0., 90. °},
 {{η0, 5. °}, 0, 90. °},
 {{Dψ0, 10 // radPerSec}, 0, 300. // radPerSec},
 {{rI31, 1}, 0, 10},
 {{tL, tLim / 10}, 0, tLim}]
```

Out[113]=



Animate *SO*(3) Solution

The animation is convincing. We have changed the cyan circle and red dot to track the plane and location of the center of mass because the angle η will become meaningless as we later go to $SU(2)$. We have also chosen initial conditions to produce an extreme amount of nutation, stressing the solution.

Things to Try

- Set the initial spin speed, `Dψ0`, to zero. Watch the motion devolve to the familiar, freely falling pendulum. See the conserved Z component, p_z, of the angular momentum go to zero.
- Observe non-zero p_z for other values of `Dψ0`, as spin angular momentum "leaks" into p_z, which is still conserved.
- Slide δ_0 to 90 ° and watch the equations of motion diverge. `Sec[δ]` goes to `ComplexInfinity`.
  - *Any other choice of Euler angles would fail somewhere. To circumvent this failure, we must avoid Euler angles. In an upcoming notebook, we will abandon trigonometry entirely and lift the geometry into SU(2), trading singularities of SO(3) for a seamless algebra of spinors / quaternions.*
- Set high values for spin speed or I_3 to observe total resistance to falling, but with slow precession and small-amplitude nutation.

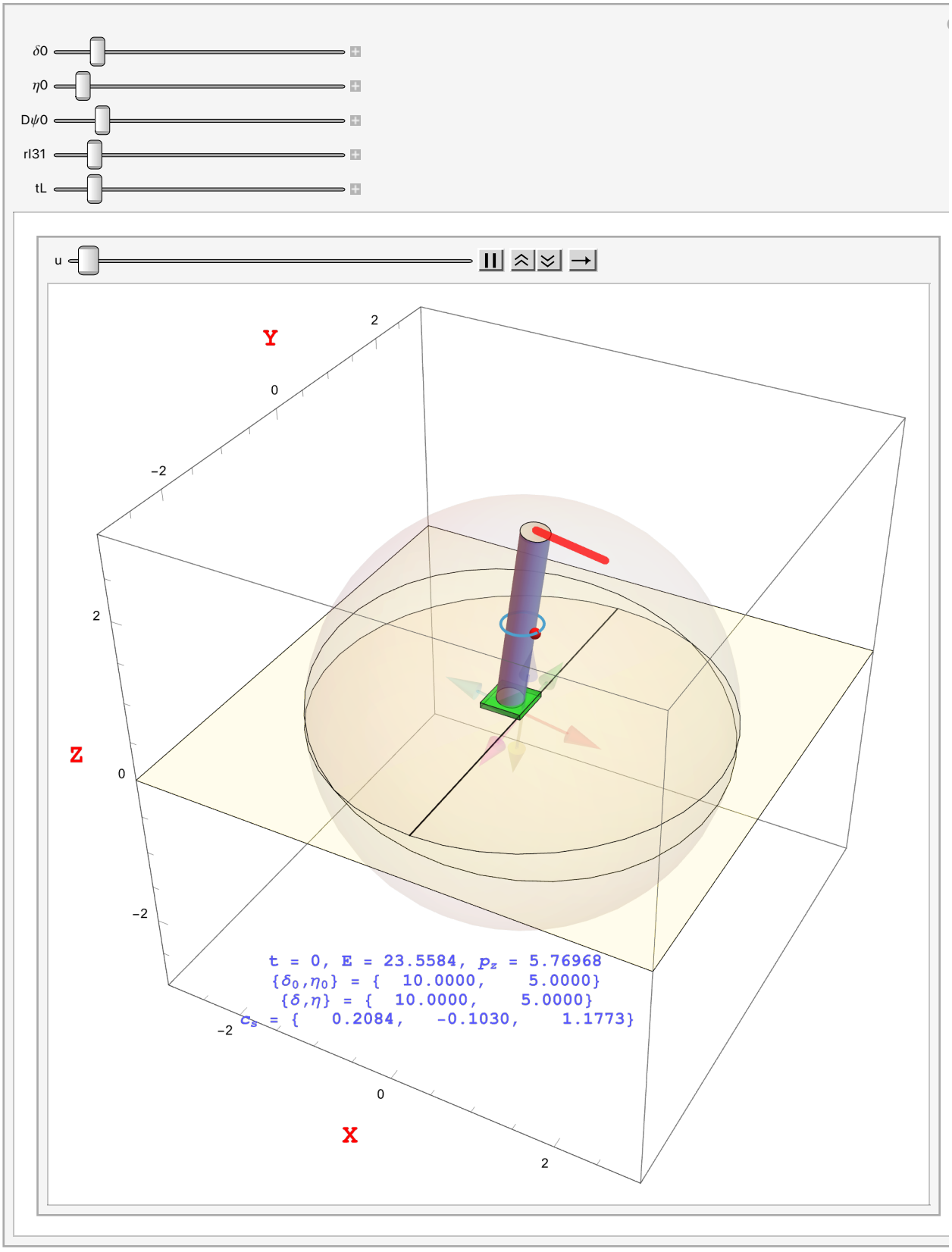
In[114]:=

```
ClearAll[nfm, vfm, font, myFont];
myFont[color_, size_ : 18] :=
  Style[#, color, Bold, size, FontFamily → "Courier New"] &;
nfm[num_, dig_ : 10, dec_ : 4] := ToString[NumberForm[Chop@num, {dig, dec}]];
vfm[vec_] := ToString[
   NumberForm[#, {7, 4}, NumberSigns → {"-", " "}, NumberPadding → {" ", "0"}] & /@
    Chop@vec];
font = myFont[Lighter[Blue], 12];
With[{epsilon = 0.0001, h = 1 / 15., w = 1 / 4., vu = 3.},
 {ztxt = -1.5, xtxt = 0, ytxt = -2, znl = 0.3},
 {kart = Cuboid[{-w, -w, epsilon}, {w, w, epsilon + h}],
  floor = Polygon[{{-vu, -vu, 0}, {vu, -vu, 0}, {vu, vu, 0}, {-vu, vu, 0}}],
  axisLabelStyle = text ↦ Style[text, Red, Bold, 18, FontFamily → "Courier New"],
  cb = rig["cb"], thin = {0, 0, .0001}, s = 2, o = {0, 0, 0}},
 {c3 = Cylinder[{o, thin}, s cb[[3]]],
  b3 = Sphere[o, s cb[[3]]],
  t3x = Tube[{-s cb[[3]] e2, s cb[[3]] e2}, .01]},
 Manipulate[
  DynamicModule[{
    ics = {δ[0] == δ0, η[0] == η0, ψ[0] == 0, δ'[0] == 0, η'[0] == 0, ψ'[0] == Dψ0},
    δx, ηx, ψx, Dδx, Dηx, Dψx, flt, numRules},
   (*Hardcoded I1 into the I3 rule so /. resolves it in one pass*)
   numRules = {m → 1, cz → 1.2, I1 → 1.2^2, I3 → rI31 * 1.2^2};
   soln3 = NDSolve[Append[eqns3, ics] //. numRules,
     {δ[t], η[t], ψ[t], δ'[t], η'[t], ψ'[t]}, {t, 0, tLim}];
   δx = soln3[[1, 1, 2]]; Dδx = soln3[[1, 4, 2]];
```

```
ηx = soln3[[1, 2, 2]]; Dηx = soln3[[1, 5, 2]];
ψx = soln3[[1, 3, 2]]; Dψx = soln3[[1, 6, 2]];
flt = "δ₀ = " <> ToString[δ0] <>
  "°, η₀ = " <> ToString[η0] <> "°, ψ'₀ = " <> ToString[Dψ0];
Animate[
 Module[{
   renderPos = o, cs, rotfn, pz, TSO3, VSO3, EnumSO3, radius,
   δ = δx /. t → u, η = ηx /. t → u, ψ = ψx /. t → u,
   Dδ = Dδx /. t → u, Dη = Dηx /. t → u, Dψ = Dψx /. t → u},
  cs = RotationMatrix[η, e1].RotationMatrix[δ, e2].cb;
  renderPos = -cs[[1]] e1 - cs[[2]] e2;
  rotfn = RotationTransform[η, e1] @*
    RotationTransform[δ, e2] @* RotationTransform[ψ, e3];
  TSO3 = 1 / 2 ((I1 /. numRules) * (Dδ^2 + Dη^2 * Cos[δ]^2) +
      (I3 /. numRules) * (Dψ + Dη * Sin[δ])^2);
  VSO3 = (m /. numRules) * 9.81 * (cz /. numRules) * Cos[δ] * Cos[η];
  EnumSO3 = TSO3 + VSO3;
  pz = ((I1 (Dδ Sin[η] - Dη Sin[δ] Cos[δ] Cos[η]) +
       I3 (Dψ + Dη Sin[δ]) Cos[δ] Cos[η]) //. numRules);
  radius = Norm[{cs[[1]], cs[[2]]}];
  Show[{Graphics3D[{
      Text[font["t = " <> nfm@u <>
         ", E = " <> nfm@EnumSO3 <>
         ", pz = " <> ToString@NumberForm[Chop[pz, 10^-5], 6]],
       {xtxt, ytxt, ztxt - 2 znl}],
      Text[font["{δ₀,η₀} = " <> vfm@{δ0 / °, η0 / °}], {xtxt, ytxt, ztxt - 3 znl}],
      Text[font["{δ,η} = " <> vfm@{δ / °, η / °}], {xtxt, ytxt, ztxt - 4 znl}],
      Text[font["cs = " <> vfm@cs], {xtxt, ytxt, ztxt - 5 znl}],
      {Black, t3x},
      {White, Opacity[.7 / 8], c3, b3, GeometricTransformation[c3, rotfn]},
      GeometricTransformation[jack[0], rotfn],
      {Red, Sphere[cs, 1 / 16.]},
      {Yellow, Opacity[.3 / 4], floor},
      {Green, Opacity[.6], Translate[kart, renderPos]},
      {White, Opacity[.75], Translate[Translate[
         GeometricTransformation[{
           Cylinder[{{0, 0, -1.2`}, {0, 0, 1.2`}}, 0.16`],
           If[Mod[Floor[10 u], 2] == 0, Red, Green],
           Thickness[0.01], Line[{{0, 0, 1.225}, {0.8, 0, 1.225}}]}, rotfn],
         cs], renderPos + (epsilon + h) e3]}},
     ImageSize → Large,
     Axes → True,
     AxesLabel → axisLabelStyle /@ {"X", "Y", "Z"},
     PlotRange → {{-vu, vu}, {-vu, vu}, {-vu, vu}}],
```

```
      ParametricPlot3D[{radius Cos[ha], radius Sin[ha], cs[[3]]}, {ha, 0, 2 π}]}]],
  {u, 0, tL},
  AnimationRate → .5]],
{{δ0, 10. °}, 0., 90. °},
{{η0, 5. °}, 0, 90. °},
{{Dψ0, 39 // radPerSec}, 0, 300 // radPerSec},
{{rI31, 1}, 0, 10},
{{tL, tLim / 10}, 0, tLim}]]
```

Out[119]=


δ0
η0
Dψ0
rl31
tL
u
Y
2
0
−2
2
Z
0
−2
−2
0
2
X
t = 0, E = 23.5584, p_z = 5.76968
{δ_0,η_0} = { 10.0000, 5.0000}
{δ,η} = { 10.0000, 5.0000}
c_s = { 0.2084, -0.1030, 1.1773}

Notes on the Term “Gauge”

The term “gauge” is used in physics in at least five different ways:

1. Gauge Choice

- in electrodynamics (Reference 8), the arbitrary smooth scalar function $\Lambda(x^\mu)$, representing the local choice of “zero phase” at every point of spacetime. i.e., the origin of the reference frame in which the phase correction is measured
- in this notebook, the specific choice to place the poles of the coordinate system on the Equator

2. Gauge Transformation

- in electrodynamics (Reference 8), the unitary operator $e^{i q \Lambda}$ that multiplies the matter field
- in this notebook, the rotation matrix $R_X(\eta)\, R_Y(\delta)$, which you can see as `rotfn` in the visualization code.

3. Gauge Potential, Gauge Connection

the geometric field that absorbs inhomogeneous terms generated from partial derivatives across changing reference frames.

- in electrodynamics (Reference 8), the covector field A_μ, the photon
- in this notebook, off-diagonal metric component $g_{\eta\psi} = I_3 \sin\delta$

4. Gauge Covariant Derivative

- in electrodynamics (Reference 8), $D_\mu \equiv \partial_\mu - i\, q\, A_\mu$
- in this notebook, as applied to velocity to absorb the free index μ: $\nabla_\nu \dot{x}^\mu \equiv \frac{\partial \dot{x}^\mu}{\partial x^\nu} + \Gamma^\mu_{\nu\lambda}\, \dot{x}^\lambda$

5. Gauge Group

- in electrodynamics (Reference 8), $U(1)$
- in this notebook, $SO(3)/SO(2)$ and $SO(3)$

Conclusion

In this notebook, we demonstrate that the classical mechanics of rigid bodies can be well understood as a geometric gauge theory. By defining a kinetic-energy metric for the configuration space of a spinning baton, the $\mathcal{UD}$ calculus automatically generates the covariant derivatives and connections necessary to produce the correct equations of motion. Bypassing the traditional scalar energy functionals of Lagrangian mechanics, we reveal that inertial forces, such as centrifugal and Coriolis, are simply the geometric consequence of moving through a curved coordinate space.

Furthermore, we identify the exact mechanism of gyroscopic precession: the off-diagonal metric term ($I_3 \sin\delta$) acts as a gauge connection, “leaking” intrinsic spin into the pitch and roll coordinates.

However, as evinced by the breakdown of numerics at $\delta = 90\,°$, covering $SO(3)$ with a global 2D coordinate grid is impossible. Because $SO(3)$ is a non-trivial fiber bundle, Euler angles inevitably fail, resulting in gimbal lock.

To construct a global, singularity-free simulation, we will abandon Euler angles and lift the geometry into the simply connected topology of $SU(2)$. By upgrading the configuration space to spinors / quater-

nions, we will trade coordinate singularities for smooth polynomial algebra. The classical spinning top places us on the mathematical doorstep of the quantum Standard Model.

References

1. Beckman, Brian. *From the steam age to quantum fields: a unified approach via UD tensorial calculus*. Wolfram Community post.
2. Beckman, Brian. *General relativity in the UD calculus*. Wolfram Community post.
3. Beckman, Brian. *Formal Differential Geometry in the UD Calculus: Part 1*. Wolfram Community post.
4. Beckman, Brian. *Covariant Derivatives and Connections in the $\mathcal{UD}$ Calculus*. Wolfram Community post.
5. Beckman, Brian. *Riemann Curvature in $\mathcal{UD}$ + Compiler POC*. Wolfram Community post.
6. Beckman, Brian. *Schwarzschild Orbits in the $\mathcal{UD}$ Calculus*. Wolfram Community post.
7. Beckman, Brian. *Schwarzschild Metric from Scratch via $\mathcal{UD}$ Compilation*. Wolfram Community post.
8. Beckman, Brian. *Gauge Theory in $\mathcal{UD}$*. Wolfram Community post.
9. Manfredo P. do Carmo, *Differential Geometry of Curves and Surfaces, Revised and Updated Second Edition*,

   Page 286, Poincaré's Theorem: *the sum of the indices of all singularities of a differentiable vector field with isolated singular points on a compact surface equals the Euler-Poincaré characteristic of the surface.* Because the Euler characteristic of a 2D sphere is $\chi(S2) = 2$, which is not zero, it is impossible to draw a continuous tangent vector field without at least one singularity.
10. John Baez, Javier P. Muniain, *Gauge Fields, Knots and Gravity*, Part II, "Gauge Fields" and the sections on Fiber Bundles.
11. Jerrold E. Marsden, *Tudor Ratiu, with Ralph Abraham. Manifolds, Tensor Analysis, and Applications*, Third Edition (link unavailable).

    Theorem 8.5.12 (Hairy Ball Theorem). *Every vector field on an even-dimensional sphere has a critical point.*
12. William L. Burke, *Applied Differential Geometry*.